

Data Modeling

Monday, June 23, 2025 9:20 AM

Introduction to Data Modeling

What is Data Modeling?

▮ Data modeling is the structured process of creating a **visual representation (schema)** of data, showing how different entities relate to one another. It defines:

- **Data structure** (tables, columns, data types)
- **Relationships** (foreign keys, associations)
- **Rules** (constraints, validations)

It ensures that data is stored in a **consistent, efficient, and usable** form throughout the lifecycle—from ingestion to reporting.

Why is Data Modeling Important for Data Engineers?

Purpose	Benefit
Performance	Efficient queries and storage design
Data Consistency	Enforces standards and rules
Collaboration	Shared understanding among teams
Scalability	Prepares for large, complex datasets
Compliance	Ensures regulatory and audit requirements are met

Where is Data Modeling Used?

- Data Warehouses (Snowflake, Redshift, BigQuery)
- Data Lakes and Lakehouses (Delta Lake, Apache Iceberg)
- OLTP systems (e.g., banking apps)
- BI tools (e.g., Power BI, Tableau)
- ETL/ELT pipelines (e.g., dbt, Glue)

Types of Data Models

Model Type	Purpose	Stakeholders
Conceptual	High-level view of entities and links	Business, analysts
Logical	Includes data types, keys, relationships	Architects, data engineers
Physical	DB-specific schema (indexes, partitions)	DBAs, infrastructure team

Example Progression:

"Customer orders Product" (conceptual)

→ Tables: Customer(id, name), Order(id, cust_id) (logical)

→ Partitioned Order table in Redshift with indexes (physical)

OLTP vs OLAP Data Models

What is OLTP (Online Transaction Processing)?

- OLTP systems are designed for managing **real-time, transactional data**.
- Example: Banking systems, e-commerce transactions, flight bookings.
- Focus: **Fast read/write, data integrity, and concurrency**.

Key Features of OLTP:

Feature	Description
High transaction volume	Thousands of short online transactions
Real-time operations	Insert, update, delete happen instantly
Normalized schema	Reduces data redundancy (3NF or higher)
Row-oriented storage	Optimized for row-level operations
ACID-compliant	Ensures Atomicity, Consistency, Isolation, Durability

Example: E-commerce OLTP Schema

- Customer(customer_id, name, email)
- Order(order_id, customer_id, order_date)
- Product(product_id, name, price)
- OrderItem(order_id, product_id, quantity)

What is OLAP (Online Analytical Processing)?

- OLAP systems are designed for **complex analysis and read-heavy operations** on historical data.
- Example: Business intelligence dashboards, sales forecasting, analytics.
- Focus: **Aggregation, filtering, fast reads, and data summarization**.

Key Features of OLAP:

Feature	Description
Low transaction volume	Fewer transactions, but more complex queries
Read-intensive	Mostly SELECT operations
Denormalized schema	Star or Snowflake schema for speed
Columnar storage	Optimized for aggregations
Time-variant data	Stores historical data for trend analysis

Example: OLAP Data Warehouse Schema (Star)

- **Fact Table:**
sales_fact(sale_id, product_id, customer_id, amount, date_id)
- **Dimension Tables:**
product_dim(product_id, name, category)
customer_dim(customer_id, name, location)
date_dim(date_id, day, month, year)

OLTP vs OLAP – Summary Comparison

Feature	OLTP	OLAP
---------	------	------

Purpose	Day-to-day operations	Analytical reporting and trends
Users	End-users, apps	Analysts, BI teams
Queries	Simple reads and writes	Complex queries, aggregations
Schema Design	Highly normalized (3NF+)	Denormalized (Star/Snowflake)
Data Volume	Operational (recent, small)	Historical (large volumes)
Speed	Fast write and quick lookup	Fast read for batch/aggregation
Example	Insert new order	Monthly sales by product and region
Tools	MySQL, PostgreSQL, MongoDB	Redshift, BigQuery, Snowflake, Hive

What is Normalization?

Normalization is the process of **structuring a relational database** to minimize data redundancy and improve data integrity.

It organizes data into **multiple related tables** by applying rules called **normal forms**.

Goals of Normalization:

- Remove duplicate data (redundancy)
- Ensure data dependencies make sense (integrity)
- Improve consistency during inserts, updates, deletes (avoiding anomalies)

Normal Forms (Rules of Normalization):

Normal Form	Rule	Example
1NF (First Normal Form)	All values in a column must be atomic (no multiple values in a field).	tags = "food, travel" → Should be separate rows
2NF (Second Normal Form)	1NF + no partial dependency on a composite key.	Break out repeating groups into new tables
3NF (Third Normal Form)	2NF + no transitive dependency.	Non-key columns must depend only on the primary key
BCNF (Boyce-Codd Normal Form)	Stronger version of 3NF; every determinant must be a candidate key.	Advanced use cases
4NF and beyond	Deals with multi-valued dependencies	Not commonly used in most OLTP or OLAP systems

Example: Normalizing a Sales Table

Original (un-normalized):

OrderID	CustomerName	ProductName	ProductPrice
1	Alice	Apple	1.0
2	Bob	Banana	0.5
3	Alice	Banana	0.5

Normalized:

Customers

CustomerID	Name
1	Alice
2	Bob

Products

ProductID	Name	Price
1	Apple	1.0
2	Banana	0.5

Orders

OrderID	CustomerID	ProductID
1	1	1
2	2	2
3	1	2

What is Denormalization?

Denormalization is the process of **combining tables to reduce joins and improve query performance**—often used in OLAP systems.

- It introduces some redundancy but speeds up read-heavy operations.
- It's a **practical trade-off** between performance and data duplication.

Why and When to Denormalize:

Scenario	Reason
OLAP systems (analytics/reporting)	Frequent JOINS make normalized models slow
Data warehouses (star/snowflake schema)	Designed for fast aggregations
BI dashboards	Need fewer joins, fewer layers
Pre-aggregated reports	Avoid real-time computation costs

Example: Denormalized Sales Fact Table

OrderID	CustomerName	ProductName	Price
1	Alice	Apple	1.0
2	Bob	Banana	0.5

This eliminates the need to JOIN with Customer and Product tables at query time.

Normalization vs Denormalization Summary

Feature	Normalization	Denormalization
Goal	Minimize redundancy, ensure integrity	Improve performance, reduce joins
Used in	OLTP	OLAP
Data structure	Many related tables (normalized)	Fewer, wider tables
Pros	Consistency, easier updates	Faster reads, easier querying
Cons	Slower reads, more joins	More storage, risk of inconsistencies

Normalization is great for transaction systems, but we denormalize in analytics platforms to improve performance and reduce join complexity.

What is a Primary Key?

A **Primary Key** is a column (or combination of columns) that uniquely identifies each row in a table.

- **Cannot be NULL**
- **Must be unique**
- **Each table can have only one primary key**

Example:

```
CREATE TABLE Customers
(
  CustomerID INT PRIMARY KEY,

  Name VARCHAR(100),

  Email VARCHAR(100)
);
```

Here, CustomerID is the primary key and ensures no two customers can have the same ID.

What is a Foreign Key?

A **Foreign Key** creates a **relationship between two tables**. It is a column (or set of columns) in one table that refers to the **Primary Key** in another table.

- Ensures **referential integrity**
- Prevents invalid entries (e.g., referencing a customer that doesn't exist)

Example:

```
CREATE TABLE Orders
(
  OrderID INT PRIMARY KEY,

  OrderDate DATE,

  CustomerID INT,

  FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);
```

Here, CustomerID in Orders is a **foreign key** that refers to Customers.

Why Primary & Foreign Keys Are Important in Data Engineering

Key Concept	Benefit
Primary Key	Prevents duplicate records, enforces uniqueness
Foreign Key	Maintains relational integrity between tables
Both	Enable efficient joins , data modeling, ETL consistency

What are Constraints?

Constraints enforce **rules on columns** in a table. They protect data integrity and ensure business logic is respected.

Constraint	Description	Example
NOT NULL	Column cannot have NULL values	Name VARCHAR(50) NOT NULL
UNIQUE	Column must have unique values	Email VARCHAR(100) UNIQUE
DEFAULT	Sets a default value for a column	CreatedAt DATE DEFAULT CURRENT_DATE
CHECK	Limits values to specific condition	CHECK (Age >= 18)
PRIMARY KEY	Unique and NOT NULL	PRIMARY KEY (id)
FOREIGN KEY	References another table	FOREIGN KEY (cust_id) REFERENCES Customers(id)

Composite Keys (Multi-column Keys)

You can define **Primary or Foreign Keys** using multiple columns.

Example: Composite Primary Key

```
CREATE TABLE Enrollment (
  StudentID INT,
  CourseID INT,
  PRIMARY KEY (StudentID, CourseID)
);
```

This means the same student can't enroll in the same course twice.

Cascade Rules with Foreign Keys

Foreign keys can have **ON DELETE** and **ON UPDATE** rules:

Rule	Behavior
ON DELETE CASCADE	Deletes child rows if parent is deleted
ON DELETE SET NULL	Sets FK column to NULL if parent is deleted
ON DELETE RESTRICT	Prevents delete if child rows exist

Example:

FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
ON DELETE CASCADE

Real-World Usage in Data Pipelines

Scenario	Use of Keys
Customer-Purchase	Foreign key ensures purchase must reference valid customer
Data Warehouse (Dimension-Fact)	Fact table uses foreign keys to reference dimension tables
ETL Job Fail-Safe	Prevents loading invalid references due to FK constraints

Summary Table

Concept	Unique	Allows NULL	Purpose
Primary Key	Yes	No	Uniquely identify a record
Foreign Key	Maybe	Yes	Link between tables
Constraint	Varies	Varies	Enforce rules on columns

Entity-Relationship Diagrams (ERDs)**1. What is an ERD (Entity-Relationship Diagram)?**

An **Entity-Relationship Diagram (ERD)** is a **visual representation** of the **entities** (tables) in a database and their **relationships**.

Used during **data modeling**, ERDs help to:

- Understand business logic
- Define table structures
- Design relationships before implementation

2. Key Components of ERDs

Component	Description
Entity	A table or object (e.g., Customer, Order)
Attribute	A field/column inside an entity (e.g., Name, OrderDate)

Relationship	Link between entities (e.g., A customer places many orders)
Primary Key (PK)	Uniquely identifies a record in a table
Foreign Key (FK)	Links one table to another (references PK in another entity)

3. Types of Relationships

Relationship Type	Description	Example
One-to-One (1:1)	Each entity in A has one match in B	One employee → one ID card
One-to-Many (1:N)	One in A → many in B	One customer → many orders
Many-to-Many (M:N)	Many in A ↔ many in B	Students ↔ Courses (via Enrollment)

How M:N is modeled:

Use a **junction table** (e.g., Enrollment with student_id and course_id)

4. ERD Notation Styles

Notation	Symbol/Usage
Crow's Foot	Most popular, shows cardinality clearly
UML Notation	Used in object-oriented design
Chen Notation	More academic/diagrammatic

Crow's Foot Notation Example:

[Customer]—<places>—< [Order]

Meaning: One customer places many orders.

- | = exactly one
- 0 = zero or more
- < = many

5. ERD Example

Let's consider a simplified sales schema:

[Customer]

|— CustomerID (PK)

|— Name

[Order]

|— OrderID (PK)

└─ CustomerID (FK)

└─ OrderDate

[Product]

└─ ProductID (PK)

└─ ProductName

[OrderItem]

└─ OrderID (FK)

└─ ProductID (FK)

└─ Quantity

This example shows:

- One customer can place multiple orders.
- An order can have multiple products (via OrderItem).
- Product and Order have a many-to-many relationship.

6. ERD Tools (Free & Popular)

Tool	Notes
draw.io	Free, simple drag-drop interface
dbdiagram.io	Syntax-based ERD creation
Lucidchart	Visual design, free tier available
SQLDBM	Database-focused modeling tool
MySQL Workbench	Generates ERD from actual schema

Resources:

[IBM](#)

[Amazon](#)

[Data modeling101](#)

[datamodeling](#)